

Lab2: Hofstadter Q 序列实验报告

姓名: [黄雯佩]

学号: [PB24111630]

日期: [2025.11.21]

1. 解决方案描述

1.1 存储方案

使用BLKW #100指令在内存中分配100个字 (word) 的连续空间, 数组起始标签为Q_ARRAY, 对应内存中的固定地址, 采用基于0的索引方式: Q[1]存储在Q_ARRAY+0, Q[2]存储在Q_ARRAY+1, 以此类推

1.2 算法实现

算法按照以下步骤实现:

(1)初始化阶段:

从内存地址 x3100 读取输入值 N

如果 $N \leq 2$, 直接返回结果 1 到 x3101

初始化 $Q[1] = 1$, $Q[2] = 1$

(2)循环计算阶段 (i 从 3 到 N) :

每次循环后把计算结果存入Q[i]

(3)结果输出:

将 Q[N] 的值存储到内存地址 x3101

程序终止

1.3 数据访问方式

先通过LEA将Q_ARRAY的地址加载进入R2, 后使用STR写入, LDR访问

2. 实验结果

2.1 测试用例验证

用C语言程序给出了 $N=1\sim 100$ 时的 $Q(n)$

```
ab2 > lab2.c > main()
1  #include <stdio.h>
2
3  #define MAX_N 100 // 定义要计算的项数
4
5  int main() {
6      // 数组 q 用于存储数列, q[1] 对应第 1 项, q[2] 对应第 2 项, 以此类推
7      int q[MAX_N + 1]; // 数组大小为 MAX_N + 1, 方便直接使用 1~MAX_N 作为索引
8
9      // 初始化前两项
10     q[1] = 1;
11     q[2] = 1;
12
13     // 计算从第 3 项到第 MAX_N 项
14     for (int n = 3; n <= MAX_N; n++) {
15         // 根据递推公式计算第 n 项
16         // Q(n) = Q(n - Q(n - 1)) + Q(n - Q(n - 2))
17         int index1 = n - q[n - 1]; // 第一个索引: n - Q(n-1)
18         int index2 = n - q[n - 2]; // 第二个索引: n - Q(n-2)
19         q[n] = q[index1] + q[index2];
20     }
21
22     // 打印前 100 项结果 (每行打印 10 项, 方便查看)
23     printf("该数列的前 %d 项如下: \n", MAX_N);
24     for (int i = 1; i <= MAX_N; i++) {
25         printf("Q(%d)=%d ", i, q[i]); // 格式化输出, 每项占 4 个字符宽度, 保持对齐
26         if (i % 10 == 0) { // 每打印 10 项换行
27             printf("\n");
28         }
29     }
30
31     return 0;
32 }
33
34 PS E:\Courses Data\ICS\labs> & 'c:\Users\18206\.vscode\extensions\ms-vscode.cpptools-1.28.3-win32-x64\debug
Adapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-okbrchc5.z52' '--stdout=Microsoft-MIEn
gine-Out-ulst0ccs.zo1' '--stderr=Microsoft-MIEngine-Error-eeqlywbz.mod' '--pid=Microsoft-MIEngine-Pid-lsyfzc
30.niy' '--dbgExe=c:\x86_64-8.1.0-release-win32-seh-rt_v6-rev0\mingw64\bin\gdb.exe' '--interpreter=mi'
该数列的前 100 项如下: Q(1)=1 Q(2)=1 Q(3)=2 Q(4)=3 Q(5)=3 Q(6)=4 Q(7)=5 Q(8)=5 Q(9)=6 Q(10)=6 Q(11)
)=6 Q(12)=8 Q(13)=8 Q(14)=8 Q(15)=10 Q(16)=9 Q(17)=10 Q(18)=11 Q(19)=11 Q(20)=12 Q(21)=12 Q(22)=1
2 Q(23)=12 Q(24)=16 Q(25)=14 Q(26)=14 Q(27)=16 Q(28)=16 Q(29)=16 Q(30)=16 Q(31)=20 Q(32)=17 Q(33)
=17 Q(34)=20 Q(35)=21 Q(36)=19 Q(37)=20 Q(38)=22 Q(39)=21 Q(40)=22 Q(41)=23 Q(42)=23 Q(43)=24 Q(4
4)=24 Q(45)=24 Q(46)=24 Q(47)=24 Q(48)=32 Q(49)=24 Q(50)=25 Q(51)=30 Q(52)=28 Q(53)=26 Q(54)=30 Q
(55)=30 Q(56)=28 Q(57)=32 Q(58)=30 Q(59)=32 Q(60)=32 Q(61)=32 Q(62)=32 Q(63)=40 Q(64)=33 Q(65)=31
Q(66)=38 Q(67)=35 Q(68)=33 Q(69)=39 Q(70)=40 Q(71)=37 Q(72)=38 Q(73)=40 Q(74)=39 Q(75)=40 Q(76)=3
9 Q(77)=42 Q(78)=40 Q(79)=41 Q(80)=43 Q(81)=44 Q(82)=43 Q(83)=43 Q(84)=46 Q(85)=44 Q(86)=45 Q(87)
)=47 Q(88)=47 Q(89)=46 Q(90)=48 Q(91)=48 Q(92)=48 Q(93)=48 Q(94)=48 Q(95)=48 Q(96)=64 Q(97)=41 Q(9
8)=52 Q(99)=54 Q(100)=56
PS E:\Courses Data\ICS\labs>
```

2.2 程序执行截图

N=10

Registers		
R0	x0006	6
R1	x0009	9
R2	x3079	12409
R3	x000B	11
R4	xFFFF	65535
R5	x0000	0
R6	x0000	0
R7	x3009	12297
PSR	x8001	32769 CC: P
PC	x306D	12397
MCR	x0000	0

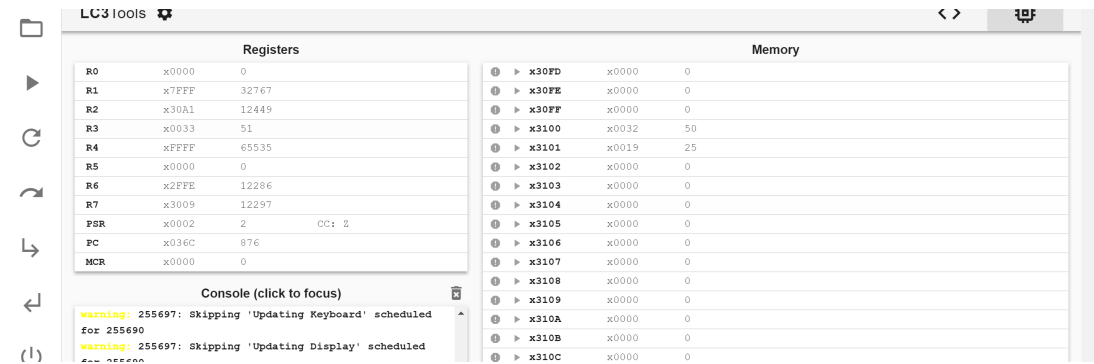
Memory		
0	x30FD	x0000 0
1	x30FE	x0000 0
2	x30FF	x0000 0
3	x3100	x000A 10
4	x3101	x0006 6
5	x3102	x0000 0
6	x3103	x0000 0
7	x3104	x0000 0
8	x3105	x0000 0
9	x3106	x0000 0
10	x3107	x0000 0
11	x3108	x0000 0
12	x3109	x0000 0
13	x310A	x0000 0
14	x310B	x0000 0

Console (click to focus)

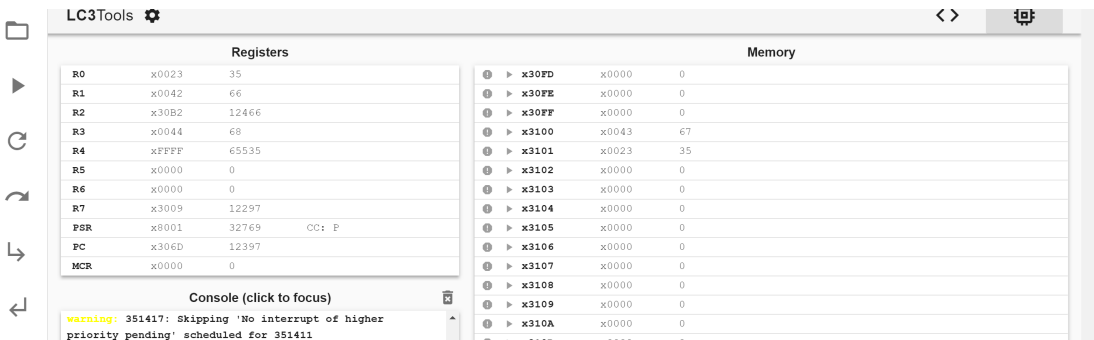
warning: 32539: Skipping 'No interrupt of higher priority pending' scheduled for 32531

warning: 32539: Skipping 'Triggering pre-instruction

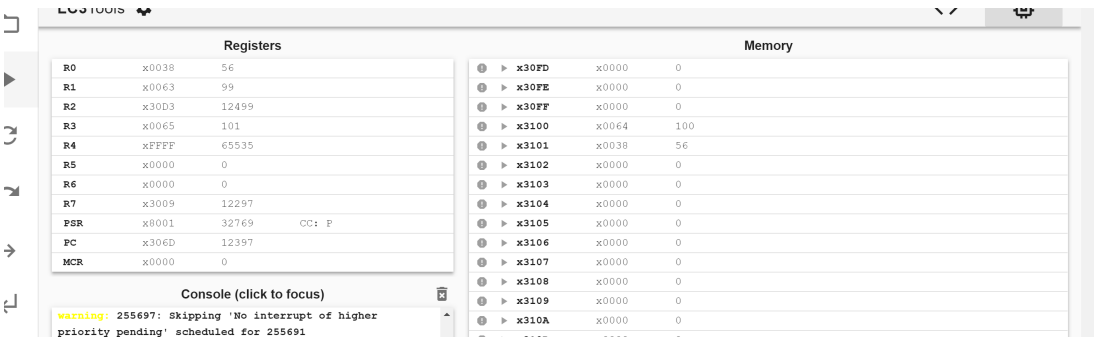
N=50



N=67



N=100



3. 挑战与解决方案

3.1 内存访问优化

参考参考方法中的优化建议，将最近计算的两个值 $Q(n-1)$ 和 $Q(n-2)$ 保存在寄存器中，减少内存访问次数。

3.2 边界条件处理

当索引值 $i-Q(n-1)$ 或 $i-Q(n-2)$ 小于 1 时，需要处理数组越界问题。在访问 $Q[t1]$ 和 $Q[t2]$ 前添加边界检查，确保索引值在有效范围内。

3.3 寄存器管理

- R0-R1: 用于临时计算和函数参数传递
- R2: 储需 Q_ARRAY 基地址
- R3: 循环计数器 i
- R5-R6: 保存 $Q(n-1)$ 和 $Q(n-2)$ 的值

4. 溢出考虑

C语言代码对前一百项的输出证明了没有溢出问题，无需取模运算